

밴디트 문제와 행동 가치 추정

탐색과 활용의 절충, 그리고 비정상 환경에서의 갱신 규칙

sajacaros
research@dacon.kr

2026년 8월 16일

초록

밴디트 문제는 에이전트의 행동이 환경의 상태를 바꾸지 않는, 강화 학습의 가장 단순한 형태다. 상태가 개입하지 않으므로 “어떤 행동이 좋은가”라는 질문만 남고, 그 결과 강화 학습의 핵심 긴장인 탐색과 활용의 절충을 다른 요소의 간섭 없이 관찰할 수 있다. 본고는 이 문제를 행동 가치 $q(A) = \mathbb{E}[R \mid A]$ 의 추정 문제로 정식화하고, 표본 평균에 기반한 추정치 $Q(A)$ 의 증분 갱신식을 유도한 뒤, ϵ -탐욕 정책을 구현하여 실험한다. 이어 보상의 확률분포가 시간에 따라 변하는 비정상 문제로 확장하여, 갱신 폭 $1/n$ 을 고정값 α 로 대체하는 것이 왜 지수 이동 평균에 해당하며 어떤 조건에서 이득인지를 보인다. 원서의 내용을 정리하는 데 그치지 않고 두 가지 보충 실험을 추가하였다. 첫째, ϵ 를 0부터 1까지 바꾸어 20,000걸음까지 관찰한 결과, 좋은 ϵ 는 문제의 성질이 아니라 플레이 길이의 함수임을 확인하였다($\epsilon = 0.01$ 이 약 7,500걸음에서 $\epsilon = 0.1$ 을 추월한다). 둘째, 환경의 변화 속도 σ 와 α 를 격자로 훑어, 고정값 α 의 우위가 α 의 크기 자체가 아니라 $\sigma > 0$ 이라는 조건에서 비롯됨을 보였다. 나아가 참값 q 를 그대로 아는 에이전트를 같은 잣대로 측정하여 성능 상한을 구하고, $\sigma > 0$ 에서 상위 α 들이 이미 상한에서 0.001 이내에 도달해 있음을 확인하였다. 이는 승률이 α 를 선택하는 잣대로 부적절하며 $|Q - q|$ 와 같은 추정 오차가 필요함을 정량적으로 뒷받침한다.

주제어: 강화 학습, 멀티암드 밴디트, 행동 가치, ϵ -탐욕 정책, 지수 이동 평균, 비정상 문제

차례

1	서론	2
1.1	머신러닝의 세 부류	2
1.2	본고의 구성과 기여	2
2	문제 설정	3
2.1	밴디트 문제	3
2.2	좋은 슬롯머신이란 무엇인가	4
2.3	행동 가치와 표기	4
3	가치 추정과 증분 구현	5
3.1	표본 평균	5
3.2	증분 구현	5
4	정책: 탐색과 활용	6
5	구현	7
5.1	환경: 슬롯머신	7
5.2	에이전트: 플레이어	7

6	실험: 정상 문제	8
6.1	한 번의 실행	8
6.2	알고리즘의 평균적 특성	8
6.3	[보충] ϵ 의 장기 거동	10
7	비정상 문제	11
7.1	표본 평균의 한계	11
7.2	고정값 α 와 지수 이동 평균	11
7.3	구현	12
7.4	표본 평균과 고정값 α 의 비교	12
7.5	[보충] α 와 환경 변화 속도 σ	13
7.6	[보충] 성능 상한과 잣대의 한계	14
8	결론	15
A	재현 정보	15

1 서론

1.1 머신러닝의 세 부류

머신러닝은 학습에 쓰이는 신호의 성격에 따라 크게 셋으로 나뉜다. 지도 학습(supervised learning)은 입력과 출력을 쌍으로 묶은 데이터로 학습하며, 각 입력에 대한 정답 레이블이 주어진다. 비지도 학습(unsupervised learning)에는 정답 레이블이 없고, 데이터에 숨어 있는 구조나 패턴을 찾는 것이 목적이다. 군집화, 특성 추출, 차원 축소가 여기에 속한다.

그림 1-1 지도 학습의 예(손글씨 숫자 '8' 학습)

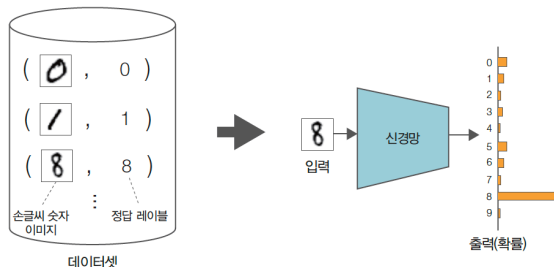


그림 1-2 비지도 학습의 예(t-SNE를 이용한 차원 축소)

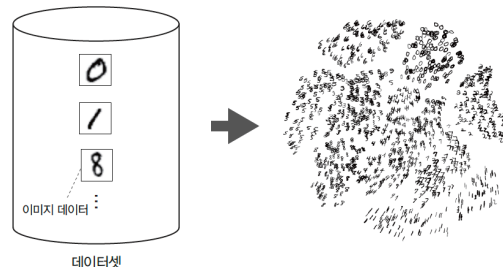


그림 1. 지도 학습(왼쪽)과 비지도 학습(오른쪽). 전자는 입력-출력 쌍으로 학습하고, 후자는 레이블 없이 데이터의 구조를 찾는다. 출처: 원서 그림 1-1, 1-2.

세 번째가 강화 학습(reinforcement learning)이다. 여기에는 정답 레이블이 없는 대신 보상이라는 신호가 있다. 에이전트가 환경에서 행동하면 환경은 보상과 상태를 돌려주고, 에이전트는 보상의 총합을 최대화하는 행동 방식, 즉 정책을 스스로 찾는다. 그림 2이 이 상호작용을 보여준다. 주목할 점은 환경이 돌려주는 것이 보상과 상태 둘이라는 사실이다. 이 중 상태가 사라지면 문제는 결정적으로 단순해지는데, 그것이 바로 본고가 다루는 밴디트 문제다.

1.2 본고의 구성과 기여

2절에서 밴디트 문제를 정식화하고 참값과 추정치의 표기를 정리한다. 3절에서 표본 평균에 기반한 가치 추정과 그 증분 구현을 유도하고, 4절에서 탐색과 활용의 절충 및 ϵ -탐욕 정책을 다룬다. 5절은 구현, 6절은 정상 문제에서의 실험이다. 7절에서 비정상 문제로 확장한다.

원서[1]의 정리에 더해 본고가 추가한 것은 다음 세 가지다.

그림 1-4 로봇 보행 문제에서의 상호작용

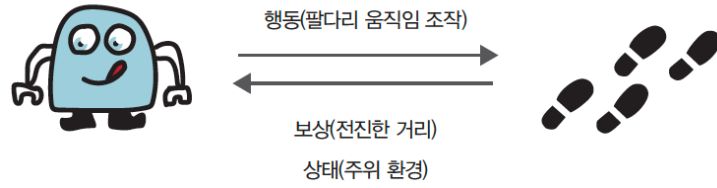


그림 2. 강화 학습에서 에이전트와 환경의 상호작용. 행동을 보내면 보상과 상태가 돌아온다. 출처: 원서 그림 1-4.

1. ϵ 의 장기 거동 (§6.3). 원서는 $\epsilon \in \{0.1, 0.3, 0.01\}$ 을 1,000걸음까지만 비교한다. 본고는 $\epsilon = 0$ 과 $\epsilon = 1$ 이라는 두 극단을 포함해 20,000걸음까지 관찰하여, ϵ 의 우열이 걸음 수에 따라 뒤집히는 지점을 특정한다.
2. α 와 σ 의 격자 탐색 (§7.5). 원서는 $\sigma = 0.1$ 인 환경에서 $\alpha = 0.8$ 하나만을 표본 평균과 견준다. 본고는 σ 를 0부터 0.1까지 바꾸어, 고정값 α 의 우위가 $\sigma > 0$ 이라는 조건에 달려 있음을 보인다.
3. 성능 상한의 측정 (§7.6). 참값 q 를 그대로 아는 에이전트를 같은 잣대로 측정하여, 승률이라는 잣대 자체가 어디서 천장에 막히는지를 수치로 제시한다.

2 문제 설정

2.1 밴디트 문제

여러 대의 슬롯머신 중 어느 것을 플레이할지 고르는 문제를 밴디트 문제라 한다. 슬롯머신을 뜻하는 속어 one-armed bandit(외팔이 강도)에서 온 이름이며, 여러 대를 동시에 다루므로 멀티암드 밴디트(multi-armed bandit) 문제라고도 부른다. 슬롯머신마다 코인이 나올 확률이 다르지만 플레이어는 그 값을 모른다. 정해진 횟수 안에 얻는 보상의 총합을 최대로 만드는 것이 목표다.

그림 1-6 밴디트 문제의 메커니즘

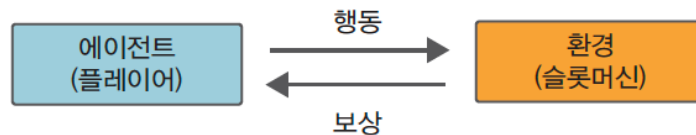


그림 3. 밴디트 문제의 메커니즘. 그림 2과 달리 상태 화살표가 없고 행동과 보상만 오간다. 출처: 원서 그림 1-6.

이 문제가 강화 학습 중 가장 단순한 형태인 이유는 에이전트가 행동해도 환경의 상태가 변하지 않기 때문이다. 상태는 늘 같으므로 사실상 존재하지 않는 것과 같고, 에이전트는 상태를 고려하지 않은 채 “어떤 행동이 좋은가”만 생각하면 된다. 그림 3에 상태 화살표가 없는 것이 그 시각적 표현이다. 문제의 구성 요소를 강화 학습의 용어로 옮기면 다음과 같다.

표 1. 밴디트 문제의 구성 요소

강화 학습의 요소	밴디트 문제에서의 대응
환경 (environment)	슬롯머신
에이전트 (agent)	플레이어
행동 (action)	어느 슬롯머신을 플레이할지 고르는 것
보상 (reward)	받은 코인
상태 (state)	없음(항상 같은 상태로 취급)

표 1에서 상태를 “없음”으로 둔 점은 강조할 만하다. 각 슬롯머신의 승률을 상태로 오해하기 쉬우나, 그것은 환경이 내부에 감춰둔 값일 뿐 에이전트가 받아 보는 신호가 아니다. 7절에서 그 승률이 시간에 따라 변하는 비정상 문제를 다루는데, 그때에도 에이전트에게 오는 것은 여전히 보상뿐이며 상태는 끝까지 등장하지 않는다.

2.2 좋은 슬롯머신이란 무엇인가

슬롯머신마다 얻을 수 있는 코인 개수의 확률분포가 정해져 있다. 그림 4의 두 슬롯머신을 보자.

그림 1-7 슬롯머신 확률분포표

슬롯머신 a					슬롯머신 b				
얻을 수 있는 동전 개수	0	1	5	10	얻을 수 있는 동전 개수	0	1	5	10
확률	0.70	0.15	0.12	0.03	확률	0.50	0.40	0.09	0.01

그림 4. 두 슬롯머신의 확률분포표. 출처: 원서 그림 1-7.

좋은 슬롯머신이란 보상의 기댓값(expected value)이 큰 슬롯머신이다. 그림 4의 값으로 계산하면

$$\mathbb{E}[R \mid A = a] = 0 \times 0.70 + 1 \times 0.15 + 5 \times 0.12 + 10 \times 0.03 = 1.05, \quad (1)$$

$$\mathbb{E}[R \mid A = b] = 0 \times 0.50 + 1 \times 0.40 + 5 \times 0.09 + 10 \times 0.01 = 0.95 \quad (2)$$

이므로 a 가 b 보다 좋다.

문제는 플레이어가 이 확률분포를 모른다는 데 있다. 알 수 있는 것은 실제로 플레이해서 받은 보상뿐이며, 따라서 기댓값을 표본으로 추정해야 한다. 그런데 표본이 적으면 추정은 쉽게 뒤집힌다. 한 번씩만 플레이해 a 에서 0, b 에서 1을 받았다면 b 가 좋아 보인다(그림 5 왼쪽). 세 번씩 플레이하면 a 의 표본 평균은 $(0+1+5)/3 = 2.0$, b 는 $(1+0+0)/3 = 0.33$ 이 되어 순위가 뒤집힌다(그림 5 오른쪽). 플레이 횟수를 늘릴수록 표본 평균은 기댓값에 가까워진다.

그림 1-8 두 대의 슬롯머신을 한 번씩 플레이하여 얻은 결과

슬롯머신	결과
	첫 번째
a	0
b	1

그림 1-9 총 세 번 플레이한 결과

슬롯머신	결과		
	첫 번째	두 번째	세 번째
a	0	1	5
b	1	0	0

그림 5. 한 번씩 플레이한 결과(왼쪽)와 세 번씩 플레이한 결과(오른쪽). 표본이 적으면 추정 순위가 쉽게 뒤집힌다. 출처: 원서 그림 1-8, 1-9.

2.3 행동 가치와 표기

보상을 확률 변수 R , 행동을 A 로 표기하자. 행동 A 를 취했을 때 받는 보상의 기댓값을 행동 가치(action value)라 하고 다음과 같이 쓴다.

$$q(A) = \mathbb{E}[R \mid A]. \quad (3)$$

앞의 예에서는 $q(a) = 1.05$, $q(b) = 0.95$ 이다. 이 표기 아래에서 밴디트 문제는 가치가 가장 큰 행동을 찾는 문제로 요약된다.

다만 에이전트는 $q(A)$ 를 모르므로 추정치 $Q(A)$ 를 만들어 써야 한다. 본고는 원서를 따라 소문자 q 를 환경이 갖고 있는 참값에, 대문자 Q 를 에이전트가 짐작한 추정치에 일관되게 사용한다. 학습이란 결국 $Q(A)$ 를 $q(A)$ 에 가깝게 만드는 일이다.

표 2. 참값, 추정치, 관측 승률의 구분

표기	무엇인가	어떻게 정해지나	누가 아는가
참값 $q(A)$	행동 가치의 실제 값 (= 진짜 승률)	환경이 처음부터 갖고 있는 값	환경만
추정치 $Q(A)$	에이전트가 짐작한 값	받은 보상으로 계산	에이전트
관측 승률	실제로 이긴 비율	보상 총합 ÷ 플레이 횟수	둘 다

5절 이후의 구현에서 보상은 0(꽂) 또는 1(당첨) 두 값만 갖는다. 이 경우

$$q(A) = \mathbb{E}[R \mid A] = P(R = 1 \mid A) \quad (4)$$

이므로 행동 가치의 참값이 곧 그 슬롯머신의 진짜 승률이 된다. 이하에서 두 표현을 섞어 쓰는 것은 이 때문이다.

3 가치 추정과 증분 구현

3.1 표본 평균

n 번 플레이해 보상 $R_1, \dots, R_n$ 을 받았다면, 행동 가치의 추정치는 표본 평균으로 둔다.

$$Q_n = \frac{R_1 + R_2 + \dots + R_n}{n}. \quad (5)$$

큰 수의 법칙에 의해 시도 횟수 n 이 커질수록 Q_n 은 참값 q 에 수렴한다.

식 (5)을 그대로 구현하면 보상을 모두 저장해 두고 매번 전체 합을 다시 계산하게 된다. 메모리와 계산량이 모두 플레이 횟수 n 에 비례해 늘어나므로 비효율적이다.

3.2 증분 구현

식 (5)은 이전 추정치만으로 갱신할 수 있는 형태로 바꿀 수 있다. $n - 1$ 번까지의 보상 총합이 $(n - 1)Q_{n-1}$ 이라는 사실

$$R_1 + R_2 + \dots + R_{n-1} = (n - 1) Q_{n-1} \quad (6)$$

을 식 (5)에 대입하면 다음을 얻는다.

$$Q_n = \frac{1}{n} \underbrace{(R_1 + \dots + R_{n-1})}_{(n-1)Q_{n-1}} + R_n \quad (7)$$

$$\begin{aligned} &= \frac{1}{n} \{ (n - 1)Q_{n-1} + R_n \} \\ &= \left(1 - \frac{1}{n}\right) Q_{n-1} + \frac{1}{n} R_n \end{aligned} \quad (8)$$

$$= Q_{n-1} + \frac{1}{n} (R_n - Q_{n-1}). \quad (9)$$

식 (9)이 증분 구현(incremental implementation)이다. 이전 추정치 Q_{n-1} 과 새로 받은 보상 R_n 만 있으면 갱신할 수 있으므로 보상을 저장할 필요가 없고, 한 번의 갱신에 드는 계산량도 n 에 무관하게 일정하다.

식 (9)의 기하학적 의미는 명확하다. 그림 6처럼 수직선 위에서 Q_{n-1} 을 R_n 쪽으로 그 거리의 $1/n$ 만큼 옮기는 연산이다. n 이 커질수록 이동 폭이 짧아지므로 갱신은 점점 둔해진다. 이 성질이 7절에서 비정상 문제를 다룰 때 결정적인 약점이 된다.

코드로는 $Q = Q + (\text{reward} - Q) / n$ 한 줄이면 된다. 예제 s01_avg.py는 식 (5)을 그대로 옮긴 기본 구현과 식 (9)의 증분 구현이 같은 값을 내놓음을 확인한다.

그림 1-10 Q_{n-1} , Q_n , R_n 의 위치 관계

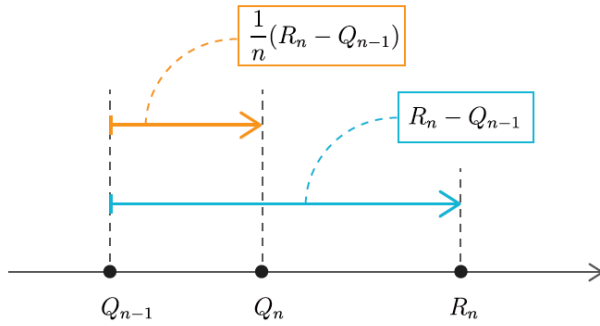


그림 1-11 대입 연산자(=)의 오른쪽과 왼쪽에 있는 Q의 실제

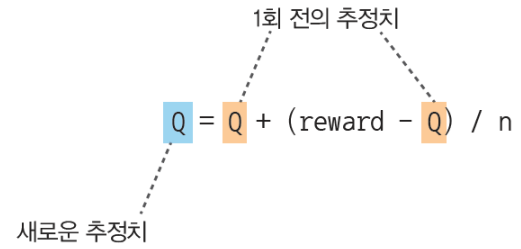


그림 6. 식 (9)의 두 가지 읽기. 왼쪽은 수직선 위에서 R_n 쪽으로 $1/n$ 만큼 이동하는 연산으로, 오른쪽은 코드 한 줄로 본 것이다. 출처: 원서 그림 1-10, 1-11.

4 정책: 탐색과 활용

추정치 Q 가 준비되면 다음 질문은 그것을 어떻게 쓸 것인가이다. 가장 단순한 답은 추정치가 가장 큰 행동을 고르는 탐욕 정책(greedy policy)으로, 지금까지 얻은 정보를 최대한 써먹는다는 뜻에서 활용(exploitation)이라 부른다. 그러나 추정치는 부정확할 수 있고, 특히 초반에 우연히 좋아 보인 슬롯머신에 갇히면 그 오해를 스스로 교정할 기회가 사라진다.

반대편에 탐색(exploration)이 있다. 추정치와 무관하게 다른 슬롯머신도 시도해 정보를 모으는 것으로, 당장의 보상은 손해지만 추정치의 정확도가 올라간다. 활용과 탐색은 한쪽을 늘리면 다른 쪽이 줄어드는 상충 관계(trade-off)에 있다.

가장 단순한 절충안이 ϵ -탐욕 정책(ϵ -greedy policy)이다. 확률 ϵ 로 무작위 행동을 취하고(탐색), 확률 $1 - \epsilon$ 로 추정치가 최대인 행동을 취한다(활용). 예컨대 $\epsilon = 0.1$ 이면 10번 중 1번은 무작위로 플레이한다.

알고리즘 1 ϵ -탐욕 정책에 의한 밴디트 학습

입력: 슬롯머신 대수 K , 탐색 확률 ϵ , 총 걸음 수 T

```

1:  $Q(a) \leftarrow 0, n(a) \leftarrow 0$  ( $a = 1, \dots, K$ )
2: for  $t = 1$  to  $T$  do
3:    $u \sim \text{Uniform}(0, 1)$ 
4:   if  $u < \epsilon$  then
5:      $A_t \leftarrow \{1, \dots, K\}$ 에서 균등하게 뽑은 행동
6:   else
7:      $A_t \leftarrow \arg \max_a Q(a)$ 
8:   end if
9:   환경에서 보상  $R_t$ 를 받는다
10:   $n(A_t) \leftarrow n(A_t) + 1$ 
11:   $Q(A_t) \leftarrow Q(A_t) + \frac{1}{n(A_t)}(R_t - Q(A_t))$ 
12: end for
  
```

▷ 탐색

▷ 활용

▷ 식 (9)

5 구현

5.1 환경: 슬롯머신

환경은 Bandit 클래스가 맡는다. 생성 시 각 슬롯머신의 승률을 $[0, 1)$ 의 무작위값으로 초기화하고, `play(arm)`은 `arm`번째 슬롯머신을 플레이해 그 승률의 확률로 1을, 아니면 0을 반환한다.

```
class Bandit:
    def __init__(self, arms=10): # arms = 슬롯머신 대수
        self.rates = np.random.rand(arms) # 각 슬롯머신의 승률 = 참값 q

    def play(self, arm):
        rate = self.rates[arm]
        if rate > np.random.rand():
            return 1
        else:
            return 0
```

코드 1. 환경 구현 (s02_bandit.py)

여기서 `self.rates[arm]`이 §2.3에서 말한 참값 $q(A)$, 곧 그 슬롯머신의 진짜 승률이다. 이 배열에 대해 네 가지를 분명히 해 둘 필요가 있다. 첫째, `Bandit()`을 만드는 순간 값이 정해지고 그 판이 끝날 때까지 바뀌지 않는다(정상 문제). 둘째, 데이터로 계산한 값이 아니라 환경이 처음부터 갖고 있는 값이며, 에이전트는 이 배열을 볼 수 없다. 에이전트에게 주어지는 것은 오직 0 또는 1의 보상뿐이고, 그것만으로 $rates(=q)$ 를 짐작한 $Qs(=Q)$ 를 만들어야 한다. 셋째, `Bandit()`을 새로 만들 때마다 승률이 다시 뿔히므로 실험마다 문제의 난이도가 달라진다. 넷째, 그럼에도 우리가 참값 q 를 알고 있는 것은 이것이 시뮬레이션이기 때문이며, 덕분에 $|Q - q|$ 같은 추정 오차를 잴 수 있다. §7.6의 성능 상한 측정도 같은 특권에 기댄다.

5.2 에이전트: 플레이어

에이전트는 각 슬롯머신의 가치 추정치 Qs 와 플레이 횟수 ns 를 들고 다니며, `update()`에서 식 (9)로 추정치를 갱신하고 `get_action()`에서 ϵ -탐욕 정책으로 행동을 고른다.

```
class Agent:
    def __init__(self, epsilon, action_size=10):
        self.epsilon = epsilon # 무작위로 행동할 확률(탐색 확률)
        self.Qs = np.zeros(action_size) # 참값 q에 대한 추정치 Q
        self.ns = np.zeros(action_size)

    # 슬롯머신의 가치 추정
    def update(self, action, reward):
        self.ns[action] += 1
        self.Qs[action] += (reward - self.Qs[action]) / self.ns[action]

    # 행동 선택(ε-탐욕 정책)
    def get_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(0, len(self.Qs)) # 무작위 행동 선택
        return np.argmax(self.Qs) # 탐욕 행동 선택
```

코드 2. 에이전트 구현 (s02_bandit.py)

6 실험: 정상 문제

6.1 한 번의 실행

슬롯머신 10대, $\epsilon = 0.1$, 1,000걸음으로 알고리즘 1을 한 번 실행한다. 한 걸음은 행동 선택 → 플레이하여 보상 획득 → 추정치 갱신의 세 단계로 이루어진다.

그림 1-12 단계에 따른 보상 총합

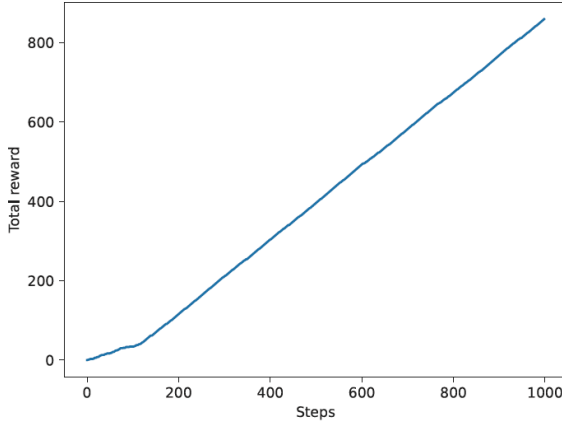


그림 1-13 단계별 승률

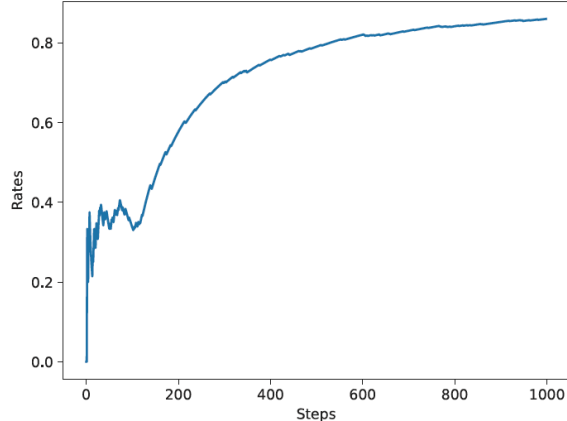


그림 7. 단일 실행의 결과. 왼쪽은 보상 총합, 오른쪽은 그 기울기에 해당하는 평균 승률 (누적 보상 ÷ 걸음 수)이다. 출처: 원서 그림 1-12, 1-13.

그림 7 왼쪽에서 보상 총합의 기울기가 곧 승률이다. 120걸음 부근에서 한 번 꺾인 뒤로는 거의 직선인데, 좋은 슬롯머신을 찾아 계속 당기고 있다는 뜻이다. 같은 정보를 승률로 그린 것이 오른쪽으로, 120걸음 부근까지는 표본이 적어 0.2에서 0.4 사이를 크게 오르내리다가, 이후 꾸준히 올라 600걸음 무렵 0.8을 넘어서고 그 뒤로는 완만하게 안정된다.

단, 슬롯머신의 승률과 플레이 결과가 모두 무작위이므로 이 그림은 실행할 때마다 달라진다.

6.2 알고리즘의 평균적 특성

같은 알고리즘을 10번 돌린 결과를 겹쳐 그리면(그림 8) 1,000걸음 시점에서 승률이 0.6대부터 0.95까지 세로로 넓게 흩어진다. 알고리즘이 같은데도 이렇게 벌어지는 것은 슬롯머신의 승률이 실험마다 다르게 뽑히기 때문이다. 따라서 한 번의 결과로는 알고리즘의 성능을 판단할 수 없다.

해법은 여러 번 실험하고 각 걸음마다 평균을 내는 것이다. 그림 8 오른쪽의 빨간 화살표가 그 방향을 보여준다. 1걸음 시점은 1걸음 시점끼리, 1000걸음 시점은 1000걸음 시점끼리 묶는다. 구현에서는 200번의 실험 결과를 (200, 1000) 형상의 배열 `all_rates`에 쌓고 `np.average(all_rates, axis=0)`으로 평균한다.

그 결과가 그림 9 왼쪽이다. 그림 7의 출렁임이 사라지고 매끄러운 곡선이 남는다. y 축이 0.5에서 시작하는 것은 아무것도 모르는 상태에서 무작위로 고른 것과 같은 승률이기 때문이며, 이 곡선이 이 알고리즘의 “평균적인 실력”이다.

ϵ 를 0.1, 0.3, 0.01로 바꾸어 비교하면(그림 9 오른쪽) 150걸음 부근에서 $\epsilon = 0.1$ 이 $\epsilon = 0.3$ 을 추월하는 교차점이 나타난다. 교차점 왼쪽에서는 탐색을 많이 하는 $\epsilon = 0.3$ 이 좋은 슬롯머신을 빨리 찾아 앞서지만, 오른쪽에서는 다 찾은 뒤에도 30%씩 더 데를 당기는 비용이 쌓여 뒤처진다. $\epsilon = 0.01$ 은 1,000걸음 내내 오르는 중이어서 아직 학습이 끝나지 않았음을 보여준다. 이 마지막 관찰이 다음 절의 출발점이다.

그림 1-14 10번의 결과를 한꺼번에 그려보기

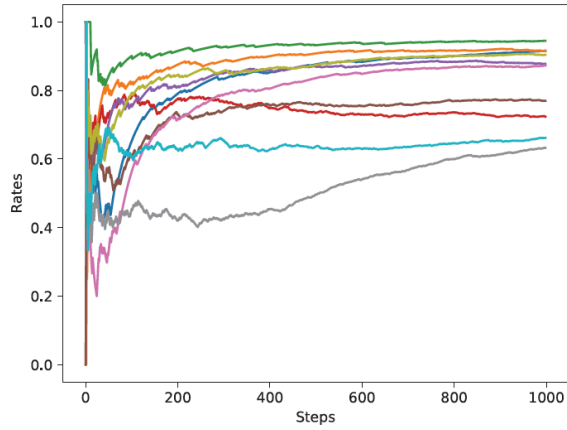


그림 1-15 단계별 평균 구하기

	1	2	3	...	1000
1번째 실험	1.0	0.5	0.333	...	0.913
2번째 실험	0.0	0.0	0.0	...	0.821
...	...	...	...	...	...
200번째 실험	1.0	1.0	1.0	...	0.615
평균	0.493	0.497	0.504	...	0.838

그림 8. 10번의 실행을 겹쳐 그린 결과(왼쪽)와 평균을 내는 방향(오른쪽). 평균은 한 실험의 시간축이 아니라 같은 걸음 수끼리 세로로 낸다. 출처: 원서 그림 1-14, 1-15.

그림 1-16 단계별 승률(200번 실험 후 평균)

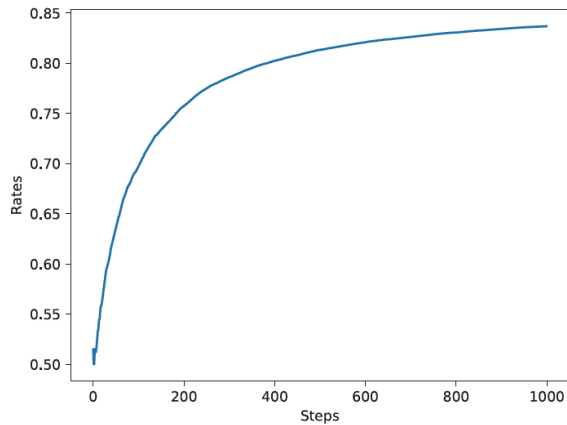


그림 1-17 ϵ -탐욕 정책의 ϵ 값을 바꾼 결과

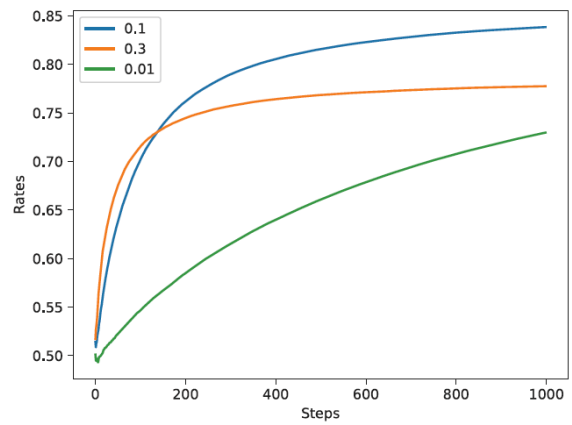


그림 9. 200번 실험의 걸음별 평균 승률(왼쪽)과 ϵ 를 세 가지로 바꾼 비교(오른쪽). 출처: 원서 그림 1-16, 1-17.

6.3 [보충] ε 의 장기 거동

원서는 $\varepsilon = 0.01$ 이 “아직 오르는 중”이라는 관찰에서 멈춘다. 그렇다면 충분히 오래 플레이 하면 $\varepsilon = 0.01$ 이 $\varepsilon = 0.1$ 을 따라잡는가? 이를 확인하기 위해 두 실험을 수행하였다¹ 첫째, $\varepsilon \in \{0, 0.01, 0.1, 0.3, 1\}$ 로 200회 실험 $\times$ 1,000걸음. 둘째, $\varepsilon \in \{0.01, 0.1\}$ 로 100회 실험 $\times$ 20,000걸음.

표 3. 정상 문제에서 ε 에 따른 평균 승률. 왼쪽 두 열은 200회 실험 $\times$ 1,000걸음, 오른쪽 네 열은 100회 실험 $\times$ 20,000걸음의 결과다. 각 열의 최댓값을 굵게 표시하였다.

ε	1,000걸음 실험		20,000걸음 실험			
	100걸음	1,000걸음	1,000걸음	5,000걸음	10,000걸음	20,000걸음
0	0.4856	0.4862	—	—	—	—
0.01	0.5630	0.7414	0.7177	0.8386	0.8667	0.8819
0.1	0.7137	0.8441	0.8344	0.8545	0.8577	0.8596
0.3	0.7111	0.7710	—	—	—	—
1	0.4882	0.4918	—	—	—	—

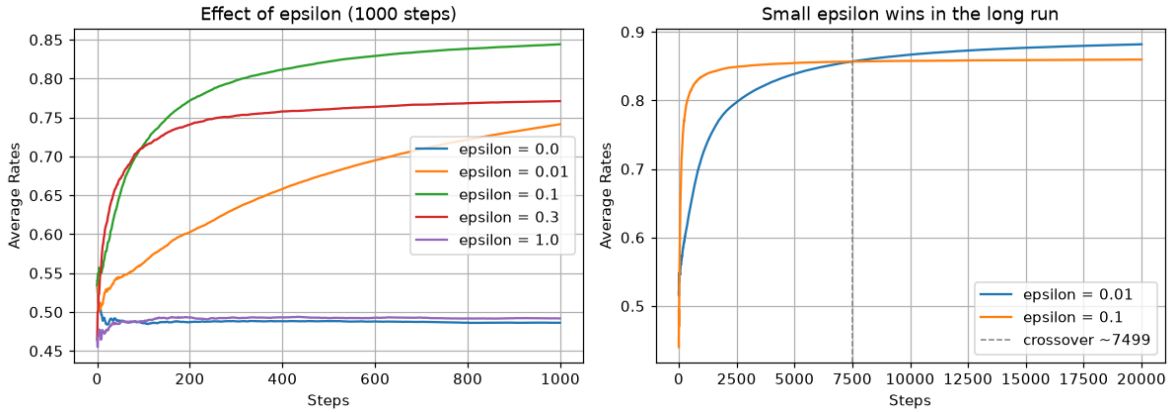


그림 10. ε 에 따른 평균 승률. 왼쪽은 1,000걸음까지, 오른쪽은 20,000걸음까지이며 회색 점선이 $\varepsilon = 0.01$ 과 $\varepsilon = 0.1$ 의 교차점이다. 난수 시드를 0으로 고정하여 재현 가능하다.

표 3과 그림 10에서 세 가지를 읽을 수 있다.

첫째, 두 극단은 모두 실패한다. $\varepsilon = 0$ (활용만)과 $\varepsilon = 1$ (탐색만)은 1,000걸음 뒤에도 각각 0.4862와 0.4918로, 무작위로 고른 것과 다를 바 없는 0.5 근처에 머문다. 실패의 이유는 정반대다. $\varepsilon = 0$ 은 처음 좋아 보인 슬롯머신에 갇혀 더 나은 것을 찾지 못하고, $\varepsilon = 1$ 은 찾아 놓고도 쓰지 않는다. 학습이 일어나려면 탐색과 활용이 함께 있어야 한다.

둘째, 1,000걸음까지는 $\varepsilon = 0.1$ 이 0.8441로 압도적이지만 그것이 최종 순위는 아니다. 20,000걸음 실험에서 $\varepsilon = 0.1$ 은 0.8596에서 사실상 포화하는 반면 $\varepsilon = 0.01$ 은 계속 올라 0.8819에 이른다. 역전이 확정되는 지점은 7,499걸음이다.

셋째, 이 역전은 비용과 이득의 시간 구조에서 나온다. 탐색에 드는 비용은 매 걸음 ε 의 확률로 일정하게 발생하지만, 탐색으로 찾아낸 이득은 남은 걸음 내내 쌓인다. 그러므로 오래 플레이할수록 작은 ε 가 유리해진다. 요컨대 “좋은 ε ”는 문제의 성질이 아니라 얼마나 오래 플레이하는가에 달려 있다.

¹실험 스크립트: s04_epsilon_comparison.py.

7 비정상 문제

7.1 표본 평균의 한계

지금까지는 보상의 확률분포가 변하지 않는 정상 문제(stationary problem)를 다루었다. 현실의 많은 문제는 시간이 지나면서 확률분포가 변하는 비정상 문제(non-stationary problem)다. 이때는 오래된 보상일수록 현재의 가치를 제대로 반영하지 못하므로 최근 보상을 더 중시해야 한다.

그런데 표본 평균은 모든 보상을 똑같은 비중으로 다룬다. 그림 11 왼쪽에서 막대 높이가 전부 같은 것이 그 표현으로, 1,000번째 보상과 첫 번째 보상의 발언권이 동일하다. 승률이 변하는 환경에서는 이미 낡은 R_1 이 현재 추정치를 계속 붙잡고 있는 셈이다.

그림 1-18 각 보상에 대한 가중치(표본 평균의 경우)

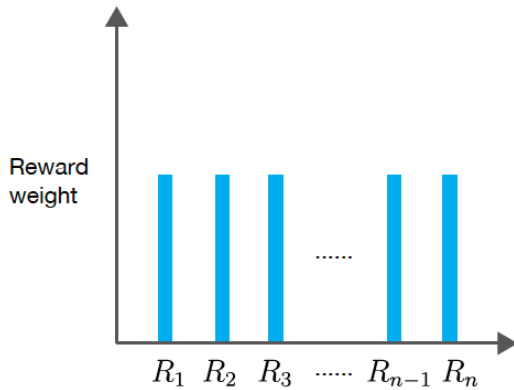


그림 1-19 각 보상에 대한 가중치(고정값 α 로 갱신한 경우)

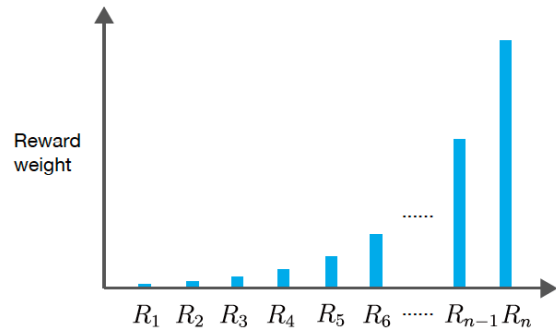


그림 11. 각 보상에 붙는 가중치. 표본 평균(왼쪽)은 모든 보상을 동일하게 다루지만, 고정값 α (오른쪽)는 최근 보상에 비중을 몰아준다. 출처: 원서 그림 1-18, 1-19.

7.2 고정값 α 와 지수 이동 평균

해법은 식 (9)의 갱신 폭 $1/n$ 을 고정값 $\alpha \in (0, 1]$ 로 바꾸는 것이다.

$$Q_n = Q_{n-1} + \alpha(R_n - Q_{n-1}). \quad (10)$$

이 식이 실제로 최근 보상에 더 큰 비중을 준다는 것은 전개해 보면 드러난다. 먼저 식 (10)을 정리하면

$$Q_n = \alpha R_n + (1 - \alpha)Q_{n-1} \quad (11)$$

이고, 같은 식을 한 단계 앞에 적용하면 $Q_{n-1} = \alpha R_{n-1} + (1 - \alpha)Q_{n-2}$ 이다. 이를 식 (11)에 대입하면

$$Q_n = \alpha R_n + \alpha(1 - \alpha)R_{n-1} + (1 - \alpha)^2 Q_{n-2} \quad (12)$$

이 되고, 이 대입을 끝까지 반복하면 다음을 얻는다.

$$Q_n = \alpha R_n + \alpha(1 - \alpha)R_{n-1} + \cdots + \alpha(1 - \alpha)^{n-1}R_1 + (1 - \alpha)^n Q_0. \quad (13)$$

즉 k 걸음 전의 보상 R_{n-k} 에는 $\alpha(1 - \alpha)^k$ 의 가중치가 붙는다. $0 < \alpha \leq 1$ 이므로 가중치는 과거로 갈수록 지수적으로 감소하며, 초기값 Q_0 의 영향도 $(1 - \alpha)^n$ 으로 함께 사라진다. 이런 갱신 방식을 지수 이동 평균(exponential moving average)이라 한다. 그림 11 오른쪽이 식 (13)의 가중치를 그린 것으로, 왼쪽과 같은 축인데 막대가 오른쪽으로 갈수록 높아진다. 감소 속도를 정하는 손잡이가 α 이며, 이 손잡이를 어떻게 돌릴지가 §7.5의 주제다.

7.3 구현

비정상 환경은 NonStatBandit이 흉내 낸다. 플레이할 때마다 모든 슬롯머신의 승률에 표준편차 σ 의 정규 노이즈를 더해 참값 q 가 조금씩 흔들리게 한다. σ 는 환경의 변화 속도이며, $\sigma = 0$ 이면 정상 문제와 같아진다. 승률에 별도의 절단을 두지 않으므로 σ 가 크면 참값이 1을 넘거나 0 아래로 내려갈 수 있다는 점은 결과를 읽을 때 유의해야 한다.

```
class NonStatBandit:
    def __init__(self, arms=10, sigma=0.1):
        self.arms = arms
        self.sigma = sigma # 참값 q가 한 걸음마다 흔들리는 정도
        self.rates = np.random.rand(arms) # 각 슬롯머신의 참값 q

    def play(self, arm):
        rate = self.rates[arm]
        self.rates += self.sigma * np.random.randn(self.arms) # 노이즈 추가
        if rate > np.random.rand():
            return 1
        else:
            return 0
```

코드 3. 비정상 환경 (s05_non_stationary.py)

에이전트 쪽은 update()만 식 (10)으로 바꾸면 된다. 플레이 횟수 ns 가 더 이상 필요 없다는 점도 눈에 띈다. 행동 선택 get_action()은 Agent와 동일한 ϵ -탐욕 정책이다.

```
class AlphaAgent:
    def __init__(self, epsilon, alpha, actions=10):
        self.epsilon = epsilon
        self.Qs = np.zeros(actions) # 참값 q에 대한 추정치 Q
        self.alpha = alpha # 고정값  $\alpha$ 

    def update(self, action, reward):
        #  $\alpha$ 로 갱신
        self.Qs[action] += (reward - self.Qs[action]) * self.alpha
```

코드 4. 고정값 α 에이전트 (s05_non_stationary.py)

7.4 표본 평균과 고정값 α 의 비교

$\sigma = 0.1$ 인 비정상 환경에서 표본 평균 방식과 $\alpha = 0.8$ 방식을 200회 실험으로 비교한 결과가 그림 12이다.

초반에는 두 곡선이 거의 겹친다. 표본 평균도 n 이 작을 때는 갱신 폭 $1/n$ 이 커서 변화를 잘 따라가기 때문이다. 그러나 200걸음 부근부터 갈라지기 시작해 끝까지 간격이 벌어진다. n 이 커지면서 $1/n$ 이 0에 가까워져 표본 평균은 사실상 학습을 멈추는 반면, 고정값 α 는 언제나 같은 보폭으로 최근 보상을 따라가기 때문이다.

다만 이 그림은 $\sigma = 0.1$ 이라는 하나의 환경에서 $\alpha = 0.8$ 이라는 하나의 값을 쓴 결과라는 점에 유의해야 한다. 이것만 보면 “ α 는 클수록 좋다”로 읽히기 쉽다. 다음 절이 그 해석을 검증한다.

그림 1-20 표본 평균과 고정값 α 에 의한 갱신 비교

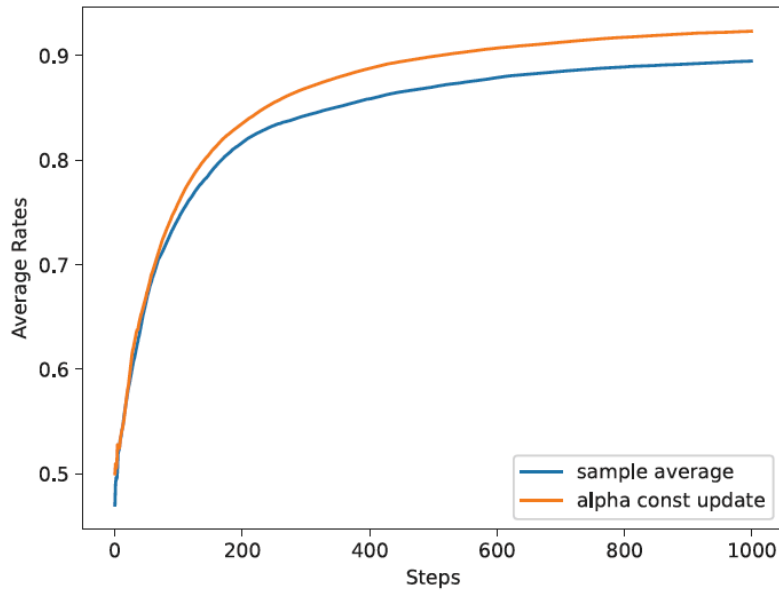


그림 12. 비정상 문제($\sigma = 0.1$)에서 표본 평균과 $\alpha = 0.8$ 의 비교. 200회 실험의 걸음별 평균이다. 출처: 원서 그림 1-20.

7.5 [보충] α 와 환경 변화 속도 σ

$\alpha \in \{0.01, 0.03, 0.1, 0.3, 0.8\}$ 와 $\sigma \in \{0, 0.01, 0.03, 0.1\}$ 의 격자에 대해, 각 조건에서 200회 실험 $\times$ 1,000걸음을 수행하고 뒤쪽 절반 구간(500–1,000걸음)의 평균 보상을 측정하였다² 앞쪽 절반을 제외한 것은 초반이 느린 작은 α 가 학습 구간 때문에 부당하게 손해 보는 것을 막기 위해서다.

표 4. σ 별 정상 구간(뒤쪽 절반) 평균 보상. 각 행에서 고정값 α 중 최고를 굵게, 표본 평균이 그 행 전체의 최고일 때 밑줄로 표시하였다. ‘상한’은 참값 q 를 그대로 아는 에이전트를 같은 조건에서 측정한 값이다 (§7.6). 행끼리의 비교는 의미가 없다.

σ	고정값 α					표본 평균	상한
	0.01	0.03	0.1	0.3	0.8		
0	0.6031	0.6712	0.8268	0.8305	0.8220	<u>0.8612</u>	0.8627
0.01	0.6306	0.7226	0.8475	0.8931	0.9052	0.8703	0.9092
0.03	0.7445	0.8099	0.9148	0.9361	0.9444	0.9000	0.9449
0.1	0.7784	0.8760	0.9347	0.9464	0.9445	0.9069	0.9474

표 4의 핵심은 각 행에서 굵게 표시한 값과 ‘표본 평균’ 열의 대소 관계다. $\sigma = 0$ 인 첫 행에서만 표본 평균(0.8612)이 모든 고정값 α 를 앞선다. 승률이 변하지 않는다면 모든 과거를 동등하게 쓰는 것이 최선이므로 당연한 결과다. 나머지 세 행에서는 고정값 α 가 이긴다. 따라서 앞 절에서 $\alpha = 0.8$ 이 표본 평균을 이긴 것은 α 가 커서가 아니라 그 환경이 $\sigma = 0.1$ 로 빠르게 변했기 때문이라고 결론 지어야 한다. 실제로 최적 α 는 σ 에 따라 이동한다 ($\sigma = 0.01, 0.03$ 에서는 0.8, $\sigma = 0.1$ 에서는 0.3).

가중치 관점에서 보면 이 이동은 자연스럽다. 그림 13 오른쪽 위에서 $\alpha = 0.8$ 의 가중치는 0.8에서 시작해 세 걸음이면 0에 수렴한다. 사실상 최근 두세 번의 보상만 기억한다는 뜻이다. 반대로 $\alpha = 0.01$ 은 0.01에서 시작해 30걸음까지 거의 평평하며(0.0100 $\rightarrow$ 0.0075), 이는 그림 11 왼쪽의 표본 평균에 가깝다. 환경이 빠르게 변할수록 짧은 기억이 유리하다.

한편 그림 13 오른쪽 아래는 성적이 아니라 추정 실력을 보는 별도의 데모다. 슬롯머신 한 대만

²실험 스크립트: s06_alpha_comparison.py.

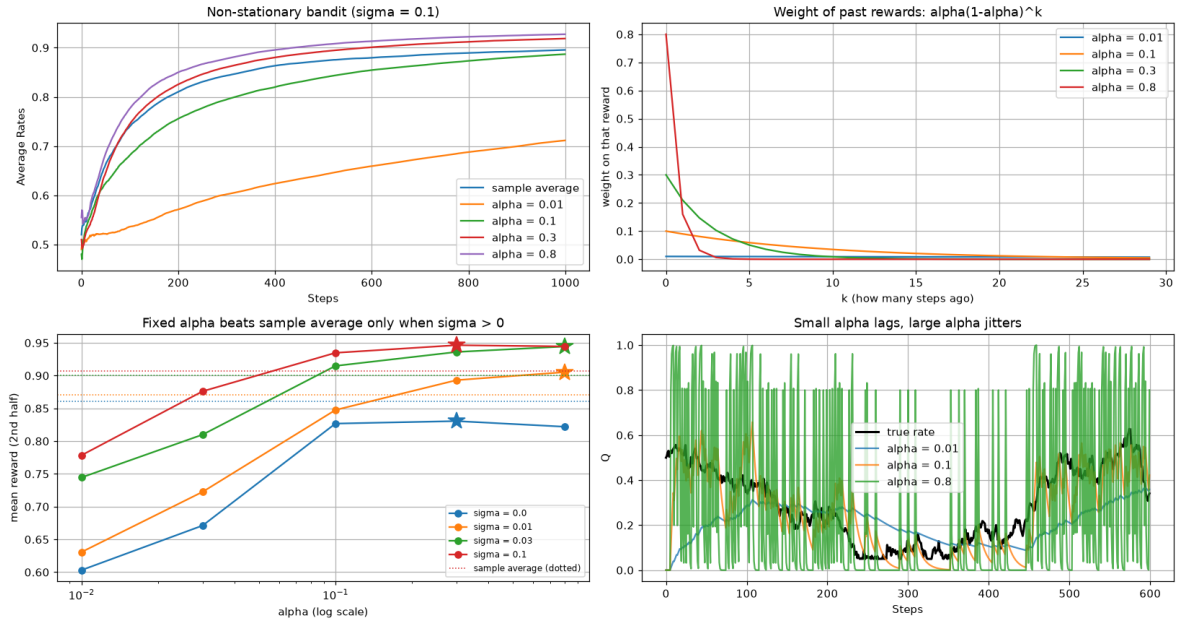


그림 13. α 비교의 네 국면. 왼쪽 위는 $\sigma = 0.1$ 에서의 승률 추이, 오른쪽 위는 과거 보상에 붙는 가중치 $\alpha(1 - \alpha)^k$, 왼쪽 아래는 σ 별 최적 α (점선은 같은 색 σ 에서 표본 평균이 낸 점수), 오른쪽 아래는 슬롯머신 한 대만 떼어내 추정치 Q 가 참값 q 를 쫓아가는 모습이다.

떼어내 선택 과정($\arg \max$ 와 ε)을 제거하고 매 걸음 그 한 대를 당기며, 세 α 가 똑같은 보상 열을 받게 하여 차이가 오직 α 에서만 오도록 하였다($\sigma = 0.02$, 참값은 $[0.05, 0.95]$ 로 절단). $\alpha = 0.01$ 은 참값이 꺾여도 한참 뒤에 따라가지만 곡선이 매끄럽고, $\alpha = 0.8$ 은 즉시 따라가지만 한 번의 보상에 통째로 휘둘러 0과 1 사이를 오간다. $\alpha = 0.1$ 이 그 중간이다. 느리고 안정적인 쪽과 빠르고 불안정한 쪽 사이의 절충이 곧 α 의 선택이다.

7.6 [보충] 성능 상한과 잣대의 한계

표 4에서 눈에 띄는 사실이 하나 더 있다. $\alpha = 0.1, 0.3, 0.8$ 세 열의 값이 거의 붙어 있어 승률 만으로는 우열을 가리기 어렵다는 점이다. 이것이 세 α 가 실제로 대등해서인지, 아니면 잣대의 문제인지를 가리기 위해 성능 상한을 측정하였다.

상한은 추정치 Q 대신 참값 q 를 그대로 보고 $\arg \max$ 를 취하는 에이전트의 점수로 정의한다. 나머지 조건은 표 4과 모두 같으며, 특히 $\varepsilon = 0.1$ 의 무작위 탐색도 그대로 남겨 같은 잣대가 되도록 하였다. 참값을 아는 에이전트를 만들 수 있는 것은 이것이 시뮬레이션이기 때문이다.

측정 결과가 표 4의 마지막 열이다. 함의는 다음과 같다.

첫째, 참값을 다 알고도 상한이 1에 미치지 못한다. $\varepsilon = 0.1$ 때문에 10번 중 1번은 무작위로 당기기 때문이며, 이것이 ε -탐욕 정책이 스스로 부담하는 비용이다.

둘째, $\sigma > 0$ 인 세 행에서 최적 α 는 상한과 0.0039, 0.0005, 0.0009밖에 차이 나지 않는다. 즉 상위 α 들은 이미 천장에 닿아 있다. 표 4의 오른쪽 세 열이 붙어 있는 것은 α 들이 대등해서가 아니라 잣대가 막혀서다. 행동을 $\arg \max_a Q(a)$ 로 고르는 이상 Q 의 순위만 맞으면 Q 가 부정확해도 받는 보상은 같으므로, 승률은 추정의 정확도에 둔감할 수밖에 없다.

셋째, $\sigma = 0$ 인 첫 행에서는 표본 평균(0.8612)이 상한(0.8627)에 사실상 도달한 반면 최적 고정값 α 는 0.0322 뒤쳐져 있다. 정상 문제에서 표본 평균이 옳은 선택이라는 앞의 결론이 여기서 다시 확인된다.

결론적으로 α 자체를 고르려면 승률이 아니라 $|Q - q|$ 와 같은 추정 오차를 재야 하며, 그것은 참값 q 를 아는 시뮬레이션에서만 가능하다. 이는 실제 응용에서 하이퍼파라미터 선택이 왜 어려운 지를 보여주는 사례이기도 하다.

8 결론

본고는 밴디트 문제를 통해 강화 학습의 기본 구도를 정리하였다. 요약하면 다음과 같다.

강화 학습은 에이전트와 환경의 상호작용에서 얻는 보상을 단서로 학습한다. 밴디트 문제는 상태가 늘 같은, 사실상 상태가 없는 가장 단순한 형태이며, 좋은 슬롯머신이란 보상의 기댓값이 큰 슬롯머신, 곧 행동 가치 $q(A) = \mathbb{E}[R | A]$ 가 큰 행동이다. 에이전트는 q 를 모르므로 추정치 Q 를 만들어 쓰며, 가치는 실제 플레이 결과의 표본 평균으로 추정하되 식 (9)의 증분 구현으로 효율화한다. 정책에서는 활용과 탐색의 균형이 핵심이고 ε -탐욕 정책이 대표적인 절충안이다. 비정상 문제에서는 갱신 폭을 고정값 α 로 두어(식 (10)) 최근 보상을 더 중시하며, 이는 식 (13)의 지수 이동 평균에 해당한다.

보충 실험에서 얻은 결론은 다음 세 가지다. 첫째, 좋은 ε 는 문제의 성질이 아니라 플레이 길이의 함수이며, 본 설정에서 $\varepsilon = 0.01$ 은 7,499걸음에서 $\varepsilon = 0.1$ 을 추월한다. 둘째, 고정값 α 의 우위는 α 의 크기가 아니라 $\sigma > 0$ 이라는 조건에서 나오며, $\sigma = 0$ 에서는 표본 평균이 옳다. 셋째, $\sigma > 0$ 에서 상위 α 들은 이미 성능 상한에서 0.001 내외에 도달해 있어 승률로는 α 를 선택할 수 없다.

다음 장에서는 에이전트의 행동이 환경의 상태를 바꾸는 문제, 즉 마르코프 결정 과정(Markov Decision Process)으로 확장한다. 상태가 도입되는 순간 “어떤 행동이 좋은가”라는 질문은 “어떤 상태에서 어떤 행동이 좋은가”로 바뀌며, 본고에서 다룬 가치 추정과 탐색·활용의 절충은 그 위에서 다시 쓰인다.

A 재현 정보

본고의 모든 수치는 다음 환경에서 재현하였다: Python 3.12.13, NumPy 2.5.2, Matplotlib 3.11.1. 보충 실험 스크립트(s04_epsilon_comparison.py, s06_alpha_comparison.py)는 np.random.seed(0)으로 난수 시드를 고정하므로 표 3과 표 4의 값은 그대로 재현된다. 코드는 다음과 같이 구성되어 있다.

표 5. 예제 코드 목록

파일	내용	출처
s01_avg.py	기본 구현과 증분 구현의 일치 확인	원서
s02_bandit.py	Bandit, Agent, 단일 실행	원서
s03_bandit_avg.py	200회 실험의 걸음별 평균	원서
s04_epsilon_comparison.py	ε 비교 (§6.3)	보충
s05_non_stationary.py	NonStatBandit, AlphaAgent	원서
s06_alpha_comparison.py	α - σ 격자와 성능 상한 (§7.5, §7.6)	보충

성능 상한 측정에 쓴 함수는 steady_rate()와 조건이 모두 같고 행동 선택만 다르다.

```
def ceiling_rate(sigma, runs=200, steps=1000):
    burn_in = steps // 2
    total = 0.0

    for _ in range(runs):
        bandit = NonStatBandit(sigma=sigma)
        reward_sum = 0

        for step in range(steps):
            if np.random.rand() < epsilon:
                action = np.random.randint(0, bandit.arms)
            else:
                action = int(np.argmax(bandit.rates)) # 참값 q를 그대로 봄
            reward = bandit.play(action)
            if step >= burn_in:
```

```
reward_sum += reward

total += reward_sum / (steps - burn_in)

return total / runs
```

코드 5. 성능 상한 측정 (s06_alpha_comparison.py)

참고문헌

- [1] 사이토 고키, 밑바닥부터 시작하는 딥러닝 4: 파이썬으로 구현하는 강화 학습 알고리즘, 한빛 미디어, 2023.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

본 문서에 실린 그림 중 “원서 그림”으로 표기한 것은 참고문헌 [1]의 도판이며, 학습 정리 목적으로 인용하였다. 그 밖의 그림과 표는 본고의 보충 실험에서 직접 생성한 것이다.